

Computer Network Defense: Wireshark Lab

Table of contents

1	Introduction	1
2	Installing Wireshark	2
3	Identifying Your Network Interface	2
4	Capturing Traffic with Wireshark	3
4.1	A Basic Capture with tshark	3
4.2	Saving a Capture to a File	4
5	Filtering Traffic	5
5.1	Capture Filters	5
5.2	Display Filters	5
6	Inspecting Packet Contents	6
7	Detecting a Port Scan	7
8	Wrap-Up Discussion	8

1 Introduction

Wireshark is a network protocol analyzer. That is, it is a tool that captures and inspects network traffic in real time. It allows you to see exactly what data is moving across a network interface on your machine: every packet, its source and destination, the protocol used, and its contents.

Network sniffing is both an offensive and defensive technique. Attackers use it during reconnaissance to identify targets, services, and unencrypted data. Defenders use it to monitor for

suspicious activity, diagnose network issues, and understand what their systems are actually doing on the network.

In this lab you will install Wireshark, capture live network traffic on your VM, and use filters to isolate and analyze specific types of traffic.

i Note

All commands in this lab should be run on your **virtual machine**, not your host machine.

2 Installing Wireshark

1. Update your package list and install Wireshark.

```
sudo apt update
sudo apt install wireshark tshark
```

During installation you will be asked: *“Should non-superusers be able to capture packets?”* Select **Yes**.

2. Add your user to the `wireshark` group so you can capture traffic without `sudo`.

```
sudo usermod -aG wireshark $USER
```

⚠ Warning

You will need to **log out and back in** for the group change to take effect. Optionally, just add `sudo` before every command.

3. Verify Wireshark is installed.

```
wireshark --version
```

4. Verify tshark is installed.

```
tshark --version
```

3 Identifying Your Network Interface

Before capturing traffic you need to know which network interface to listen on.

1. List your network interfaces.

```
ip a
```

Look for an interface that has an IP address assigned. It will likely be named something like `eth0`, `enp0s3`, or `ens33`. The loopback interface `lo` (address `127.0.0.1`) is local-only traffic.

2. Note the name of your active interface. You will use it in the next section.

4 Capturing Traffic with Wireshark

The remainder of this lab uses both `tshark` and the Wireshark GUI. Feel free to use either.

4.1 A Basic Capture with tshark

1. Open two terminals.
2. In one of the terminals, start a capture on your network interface. Replace `<interface>` with the interface name you found above (e.g. `enp0s8`). This will capture 20 packets and then stop.

```
tshark -i <interface> -c 20
```

Note the flag `-i` is for interface and the flag `-c` is for count, that is, the number of packets to capture.

While it is running, generate some traffic by pinging a server from the other terminal window.

```
ping -c 5 google.com
```

3. Familiarize yourself with the output. Each line is one packet and should include the following:
 - A packet number and timestamp
 - Source IP address
 - Destination IP address
 - Protocol
 - A brief description

! Group Discussion

Look at the captured packets with your group.

1. Can you identify which packets were generated by the `ping` command? What protocol do they use?
2. Are there any packets you did not expect? Where are they coming from or going to?

4.2 Saving a Capture to a File

Rather than reading packets as they scroll past, you can save a capture to a file and analyze it afterward.

1. Create a folder to store your captures in.

```
cd ~/Documents
mkdir "packet captures"
cd "packet captures"
```

2. Start a 45-second capture and save it to a file.

```
tshark -i <interface> -a duration:45 -w capture.pcap
```

Flags:

- `-a` is used to set an **autostop** condition. Our autostop condition is a duration of 45 seconds passing.
- `-w` flag is used to indicate the file you want to **write** the capture to.

3. While it is running, generate traffic from the other terminal:

```
ping -c 10 google.com
curl http://httpforever.com
```

4. Once the capture completes, read the file back with the following command.

```
tshark -r capture.pcap
```

5. For a nicer view, open the .pcap file with wireshark by double clicking it or using the following command.

```
wireshark capture.pcap
```

5 Filtering Traffic

Wireshark's most powerful feature is its filtering system. Filters let you focus on specific traffic rather than reading every packet.

5.1 Capture Filters

Capture filters are applied during the capture itself. Only matching packets are recorded.

1. Capture only ICMP traffic (ping packets) for 15 seconds.

```
tshark -i <interface> -a duration:15 -f "icmp" -w icmp_capture.pcap
```

Flags

- `-f` flag allows us to apply a filter.
2. While the previous command is running, in another terminal, generate some pings and some non-ping traffic:

```
ping -c 5 google.com  
curl http://httpforever.com
```

3. Read back the capture and confirm only ICMP packets were recorded.

```
tshark -r icmp_capture.pcap
```

```
wireshark icmp_capture.pcap
```

5.2 Display Filters

Display filters are applied when reading a capture. All packets are recorded but only matching ones are shown.

1. Read your earlier `capture.pcap` and filter to show only DNS traffic.

```
tshark -r capture.pcap -Y "dns"
```

Or, with the wireshark GUI

```
wireshark -r capture.pcap -Y "dns"
```

2. Filter to show only HTTP traffic.

```
tshark -r capture.pcap -Y "http"
```

```
wireshark -r capture.pcap -Y "http"
```

! Group Activity

Work through the following with your group.

1. Have one group member run the following command on their VM to start a capture. If 30 seconds isn't enough time as indicated, give yourself enough time to have the remaining parts completed.

```
tshark -i <interface> -a duration:30 -w group_capture.pcap
```

2. Have the other group members SSH into that VM during the 30-second window.

```
ssh <username>@<partner_ip_address>
```

3. After the capture ends, read it back and filter for SSH traffic:

```
tshark -r group_capture.pcap -Y "tcp.port == 22"
```

4. Discuss: Can you see the SSH connection attempts in the capture? What information is visible — and what is protected by SSH encryption?

6 Inspecting Packet Contents

You can use `tshark` to print more detail about each packet, including its contents.

1. Read your capture and print full packet details for the first 5 packets.

```
tshark -r capture.pcap -c 5 -V
```

The `-V` flag enables verbose output showing every field of every protocol layer.

2. Extract just the source and destination IP addresses from the capture using fields mode.

```
tshark -r capture.pcap -T fields -e ip.src -e ip.dst
```

3. Extract source IP, destination IP, and protocol.

```
tshark -r capture.pcap -T fields -e ip.src -e ip.dst -e frame.protocols
```

! Group Discussion

1. Run `curl http://httpforever.com` in another terminal while capturing traffic, then look at the HTTP packets in the capture. Can you read the content of the webpage in the packet data? What does this tell you about unencrypted HTTP traffic?
2. Now try `curl https://google.com` and capture that traffic. Can you read the content now? Why not?
3. What kinds of sensitive information could an attacker recover by sniffing unencrypted traffic on a network?

7 Detecting a Port Scan

A port scan is often one of the first things an attacker does on a network. It also leaves a very distinctive pattern in network traffic. In this section, one group member will scan another with `nmap` while the target captures the traffic.

⚠ Warning

Only scan the VM of a group member who has agreed to it. Do not scan any other machines on the network.

1. **Target:** start a 60-second capture.

```
tshark -i <interface> -a duration:60 -w scan_capture.pcap
```

2. **Scanner:** install `nmap` and scan the target's VM.

```
sudo apt install nmap  
sudo nmap -sS <target_ip_address>
```

The `-sS` flag performs a **SYN scan**. For each of the 1,000 most common ports, `nmap` sends the first packet of a TCP handshake (a **SYN**) and looks at the reply. An open port replies with a **SYN-ACK**, while a closed port replies with a **RST** (reset). `nmap` never completes the handshake.

3. **Target:** once the capture completes, count the connection attempts (packets with the **SYN** flag set but not the **ACK** flag) that each IP address sent you.

```
tshark -r scan_capture.pcap -Y "tcp.flags.syn == 1 && tcp.flags.ack ==  
↪ 0" -T fields -e ip.src | sort | uniq -c
```

4. **Target:** see which ports your machine answered as **open**.

```
tshark -r scan_capture.pcap -Y "tcp.flags.syn == 1 && tcp.flags.ack ==  
↪ 1" -T fields -e ip.src -e tcp.srcport | sort -u
```

Compare this with the open ports **nmap** reported on the scanner's screen.

5. Optionally, open the capture in the Wireshark GUI and go to **Statistics > Conversations**, then select the **TCP** tab. Each row is a pair of addresses and ports that exchanged traffic.

! Group Discussion

1. How many connection attempts came from the scanner, compared to any other address? What makes this pattern easy to tell apart from normal traffic?
2. If you were writing a rule for an intrusion detection system, what would it look for to detect a port scan?
3. **nmap** can be told to scan very slowly, e.g. one port every few minutes (**-T1**). Why would an attacker do this, and how does it make detection harder?

8 Wrap-Up Discussion

! Group Assignment

Before you leave, discuss the **Wireshark Lab** questions in the *Computer Network Defense: Assignment* document with your group and write down your answers. Your group will submit the answers for all of the labs together, as a single PDF on Canvas, after the final lab.